Standard Type-1 Hypervisor Boot Flow

Learning from existing mechanisms to propose consistent packaging and handoff

Linux Plumbers Conference 2026

Existing Type-1 boot mechanisms

Two mature approaches solve similar boot problems with different packaging and handoff models.

ACRN

- Hypervisor launched as an EFI image
- VM payloads loaded by the EFI image before launching Hypervisor
- VM payloads described using Multiboot2 interface.

Xen

- Grub based Multiboot2 implementation used for loading xen.efi and domain modules.
- Hypervisor and domain payloads described through Multiboot2 and Device Tree

Review lens Packaging • OS loader/manager-to-hypervisor interface • firmware transition • extensibility • portability

The interoperability gap

Boot loaders, operating systems, and Type-1 hypervisors lack one shared packaging and handoff contract.

Packaging gap

No common structure consistently identifies the hypervisor, host OS, and optional VM payloads.

Metadata gap

No generic loader-to-hypervisor schema consistently describes selected VM images and boot parameters.

Integration gap

Tooling and loader changes become tied to a particular hypervisor or boot-manager mechanism.

Need: a portable packaging format and a VM-focused handoff schema

Proposals for Packaging and Loader-Hypervisor schema

Keep packaging and runtime VM description independently reviewable and implementable.

Boot Loader specification
Type-1 Hyp UKI

Defines how a Type-1 hypervisor and optional VM payloads are carried and associated in a bootable image.

Generic Type-1 DT

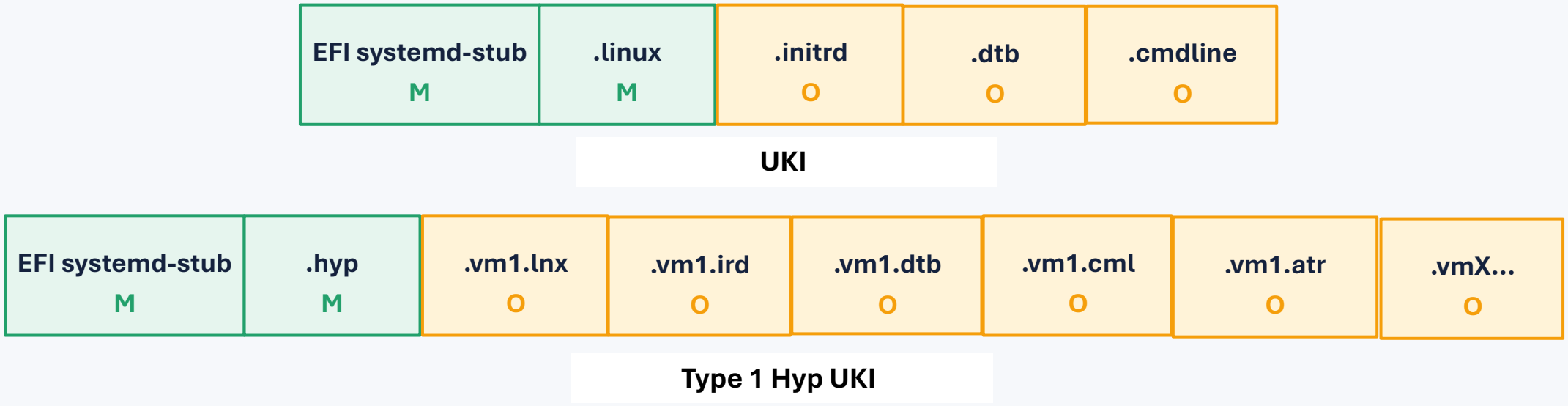
Defines the VM-related launch information passed from a loader or manager to a Type-1 Hypervisor

The two specifications can reference each other while remaining independently implementable.

Type-1 Hyp UKI packaging proposal

Introduce a mandatory hypervisor section; VM definitions and attributes may be optional.

M = mandatory O = optional



Proposed rules

- .hyp and .linux are mutually exclusive
- .hyp is mandatory for a Type-1 Hyp UKI
- VM payloads are optional

Note:

VM payload names are not final, will be discussed as part of the PR -
[RFC] boot/ukify: add Type-1 hypervisor UKI support by RakeshVakulaabharanam · Pull Request #43995 · systemd/systemd

Generic Type-1 DT: VM-focused schema

Describe only the VM information required by a Type-1 hypervisor

M = mandatory O = optional

VM

boot-order (M) = <0, 1, … n >

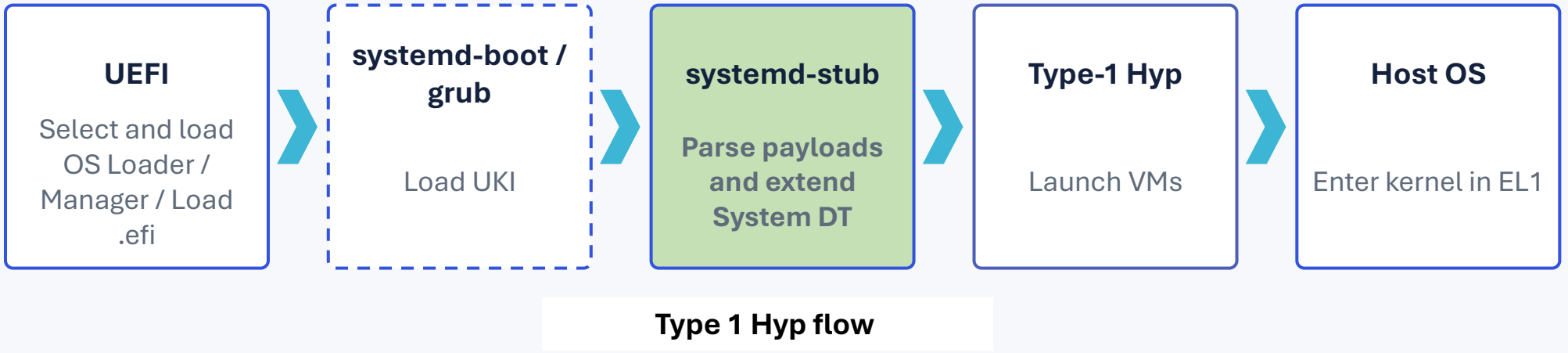
VMx x= 0, 1, … n

- VM id (M)
- start-addr (M)
- image-size (M)
- VM attributes (O)
- bootargs (O)
- initrd (O)
- dtb (O)

Field	Description
boot-order	VM launch sequence.
VM id	Unique identifier the hypervisor uses to refer to each VM.
start-addr	Physical address where the loader placed the VM image in RAM.
image-size	Size in bytes of the loaded VM image.
VM attributes	VM Auth type (Platform vs OEM), VM Type (static / pre-launch vs dynamic), etc.
bootargs	Kernel command line handed to the VM at start.
initrd	Address and size of an optional initial ramdisk.
dtb	Address of a guest device tree supplied for the VM.

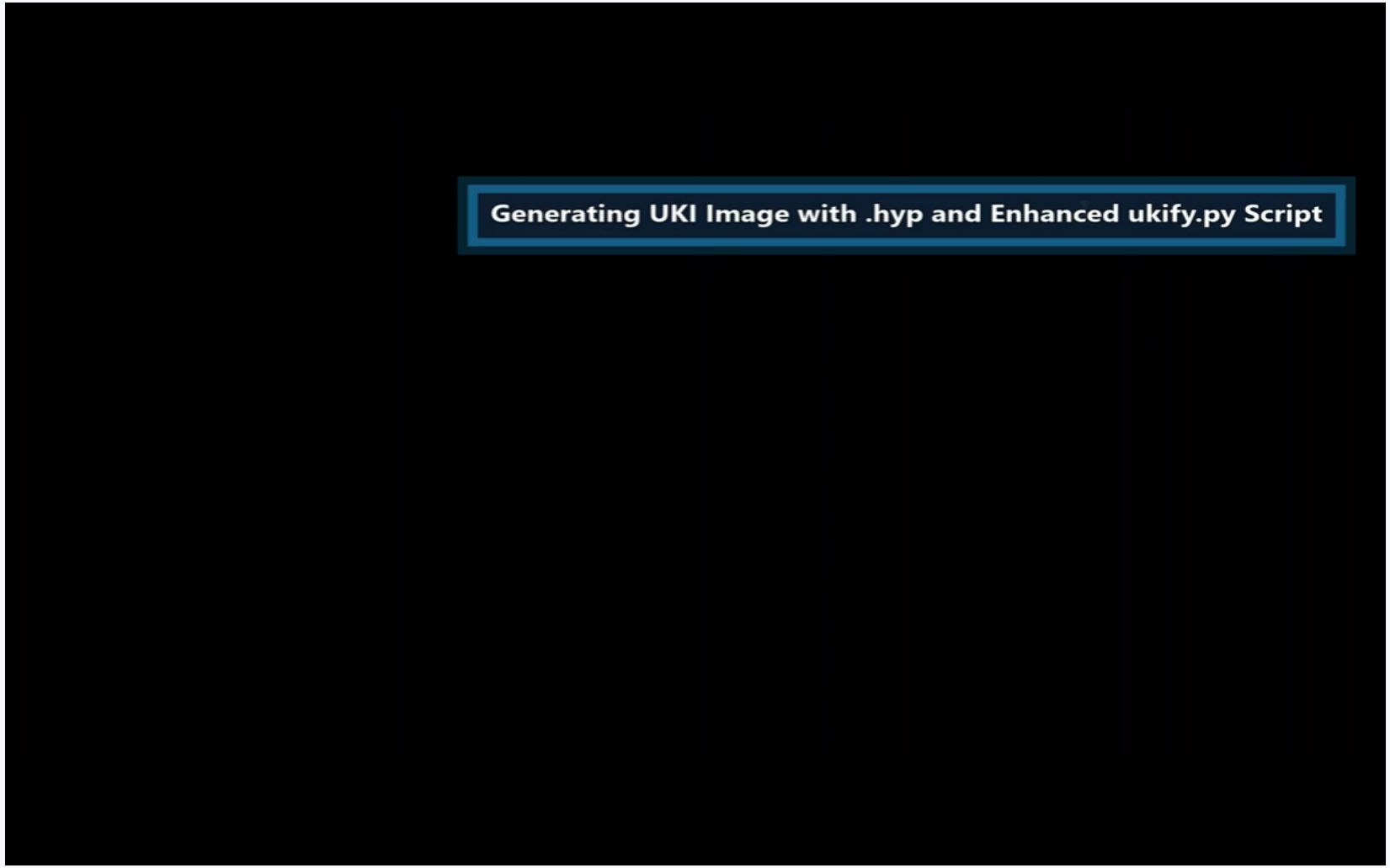
Proposed standard boot flow

Use standard firmware and loader mechanisms, then enter an EFI-aware Type-1 hypervisor.



Demo : Qualcomm SoC

Qualcomm SoC boot with UEFI->systemd-boot->systemd-stub->Qualcomm Hyp->Linux



Open questions for LPC

Use the discussion to refine scope, ownership, and tooling impact.

1. Which VM attributes are portable enough to standardize?
2. Where should the Generic Type-1 DT schema be owned and maintained?
3. How should distributions generate, deploy, and update Type-1 UKIs?
4. Is the Boot Loader Specification appropriate for the Type-1 UKI definition?

Thank You